

DSC 102 Multi-Processing(Both Task Parallel and BSP) Practice Problems and solutions

1. Suppose an SQL query takes 20 min to run on a single worker node and x min when run on 5 worker nodes. What is the speedup for the given value of x ? Is the speedup linear, sublinear, or superlinear?
 - a. $x = 7$ min
 - b. $x = 4$ min
 - c. $x = 3$ min

Ans.

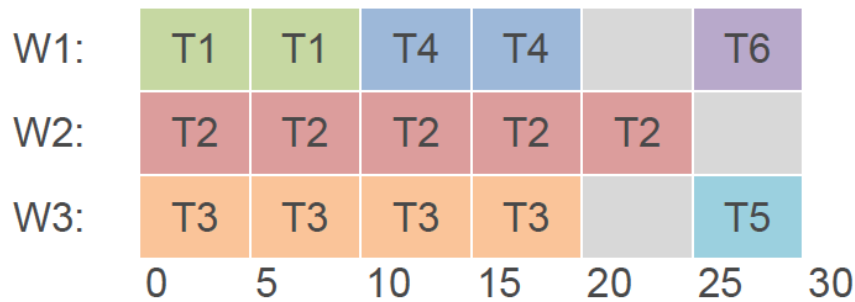
- a. *Speedup = $20/7 = 2.86$; $< 5x$, sublinear*
- b. *Speedup = $20/4 = 5$; $= 5$, linear*
- c. *Speedup = $20/3 = 6.67$; > 5 , superlinear*

2. Suppose an ML training workload takes 40 min to run on a single worker node. We then triple the dataset size, say, to help improve accuracy and use 3 worker nodes. It now takes x min. What is the speedup for the given value of x ? Is the scaleup linear, sublinear, or superlinear?
 - a. $x = 50$ min
 - b. $x = 40$ min

Ans: If it takes the same time to accomplish more work than before with added resources, then scaleup is linear.

- a. *It takes more time and the speedup is therefore less than 1 even with 3 works. Scaleup is therefore **sub-linear**.*
- b. *It can do more work in the same amount of time with additional workers which means the scaleup is **linear**.*

3. For the Gantt chart shown below compute the speedup relative to using one worker.



Ans: 3-worker time = 30 (from chart) 1-worker time = 10+25+20+10+5+5 = 75
 Speedup = 75 / 30 = 2.5

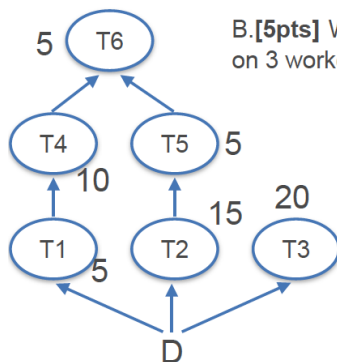
4. Suppose we now said that T6 depends only on T4 and T5 depends only on T3 and T4. Modify the Gantt chart to reflect this and compute speedup.

Ans: Ans: 3-worker time = 25 (since we can now schedule T6 and T5 on the two workers that are idle in the slot 20-25) 1-worker time = 10+25+20+10+5+5 = 75
 Speedup = 75 / 25 = 3

5. Suppose we now have 5 workers instead of 3 in Question 4. What will the speedup now be?

Ans: No different since T2 is the longest path and takes 25 units.

6. Consider the task graph shown below (ignore the text next to it):



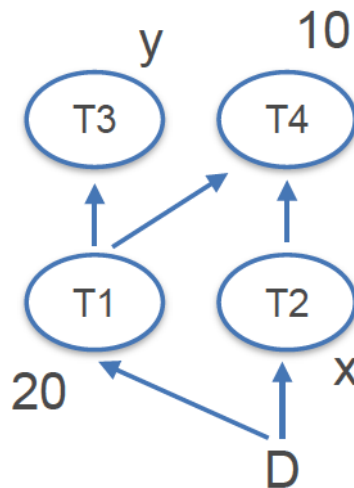
- If you had 3 workers, what is the lowest possible completion time?
- What is the highest possible speedup?
- What is the efficiency with the schedule that yields the best speedup?

Ans:

- 25 since T2, T5 and T6 form the critical path.

- b. 2.4; total 1-worker time = $5+15+20+10+5+5 = 60$ Regardless of # workers, lowest possible compl. time is 25 as above. So, highest possible speedup = $60/25 = 2.4x$
- c. Total Units consumed by 3 workers is 75 since you need at least 25 units for the critical path. However total time units used will only be 60 (single worker time. Draw the Gantt chart and figure this out). Hence Efficiency = $60/75 = 80\%$

7. Consider the following task graph with some task lengths shown and some unknown/hidden.



- a. What is the degree of parallelism in this task graph?
- b. You are given that the lowest possible completion time of this workload is 30. What are the maximum possible values of x and y?
- c. Instead of B, you are now given that $y = 2x$. What is the highest possible speedup for this workload with task-parallel execution?

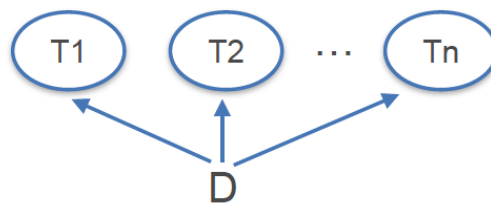
Ans:

- a. 2; only 2 tasks can run concurrently based on given graph
- b. $y = 10$, $x = 20$. Lowest possible compl. time is the longest path, i.e., $\max(20+y, 20+10, x+10)$. Set it to 30. So, max value of $y = 30-10 = 20$ $30-20 = 10$; likewise, max value of $x = 30-10 = 20$.
- c. 1.5; lowest possible comp. time now is $\max(20+2x, 20+10, x+10)$. To get the highest possible speedup, you need its lowest possible value, which is just 30. That means $20+2x$ can be max 30, which means x can be max 5.

1-worker time = $x+y+30 = 3*(x+10)$. So, max speedup is $3*(x+10)/30$.
 Setting $x=5$ then gives us $3*15/30 = 1.5x$

8. You are given a workload with n tasks, each of length k units. You are allowed to use task parallelism as discussed in class. What is the lowest possible completion time of this workload? What should the task graph look like for that and how many workers are necessary to achieve that completion time?

Ans: Lowest compl. time is k . Every task must run in parallel. So, the task graph simply has all tasks independent of each other. Exactly n workers are necessary, since there are n tasks. Any less will raise compl. time. Any more will just sit idle.

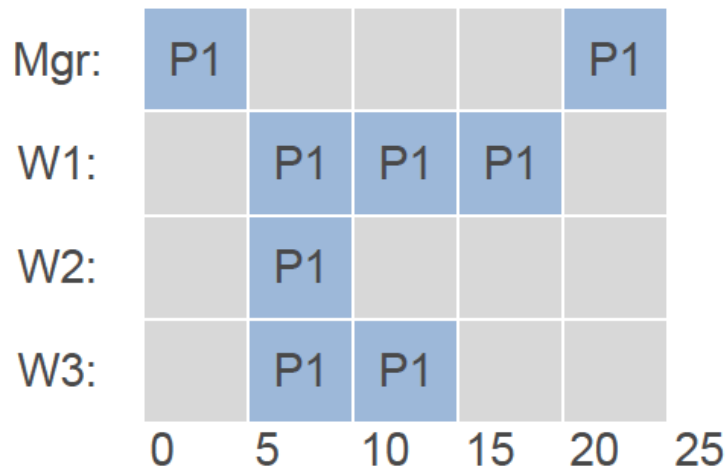


9. Suppose an ML training program has 2 parts: a parallelizable part that takes 1 hr when run with 1 core and an inherently sequential part that takes 20 min on 1 core. Suppose the parallelizable part yields perfect linear speedup on x cores with data parallelism. What is the overall speedup of the training when run on n cores? Actual runtime?
- $n = 2$
 - $n = 6$
 - $n \rightarrow \text{infinity}$

Ans: This is an application of Amdahl's law. The application is done using times instead of the concept of fraction of code that is sequential. You could do the same using the formula I gave in class where $f = 20/80$. With time the notion is the total time on one core divided by the time on multiple cores.

a. $(60+20) / (30+20) = 1.6$

b. $(60+20) / (10+20) = 2.67$



c. $(60+20) / (0+20) = 4$

10. Consider the following Gantt chart for a BSP data-parallel execution of a process on a heterogeneous 3-worker cluster.

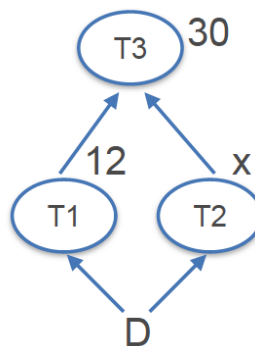
- What is the completion time of this parallel process?
- Suppose P1 took only 30 units on a single machine. What is the speedup of this 3-worker execution?
- Suggest an approach to potentially improve the speedup on this cluster.

Ans:

- 25
- $30/25 = 1.2$
- The Manager can (re)partition the data non-uniformly to allot more work to W2 than W1, perhaps matching both to W3's runtime of 10.*

Note: This problem can also be framed as a BSP problem where you are not given a Gantt chart but instead told that there is a pre-processing step of 5 time units and a post processing step of 5 time units. Further the data is split across 3 workers in the ratio 3:1:2 and processing each block of data takes 5 units of time. The solution is the same - internalize how to look at this in different ways.

11. Consider this task graph. Each task is amenable to perfect linear speedup with BSP data-parallelism, albeit with 1 unit orchestration overhead by the manager before and after each worker execution regardless of cluster size. Assume a homogeneous cluster. Given the following values of T2's length and cluster size y (manager+workers), what is the lowest possible completion time?



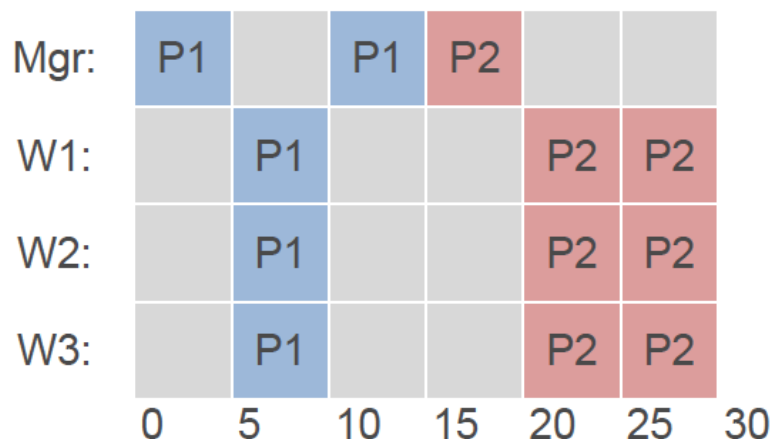
- $x = 6; y = 2$
- $x = 6; y = 3$
- $x = 6; y = 5$
- $x = 30; y = 2$
- $x = 30; y = 3$

Ans:

- For task-par. deg. of par. is 2; so runtime is $\text{Max}\{12+30, 6+30\} = 42$. For BSP, since only 1 worker exists (1 is manager), time will be higher than task-par.; likewise for hybrid. So, overall: **42**.
- For task-par., 42 again. For BSP we can parallelize work of T1, T2, and T3 perfectly across 2 workers; each will have 2 units of manager overhead. So, runtime: $3 * 2$ (Mgr only) + $12/2 + 6/2 + 30/2 = 30$. Can the hybrid model be faster than BSP? Let us try this: T1 and T2 in task-par. manner and then T3 as full BSP. Runtime now: $\text{Max}\{12, 6\} + 2$ (Mgr) + $30/2 = 29$. This hybrid is the fastest.

- c. Task-par.: 42. BSP: $3 * 2$ (Mgr only) + $12/4 + 6/4 + 30/4 = 18$. Try the hybrid from answer B: T1 & T2 in task-par. followed by T3 in BSP will be: $12 + 30/4 + 2 = 21.5$ (not 19.5, which missed 2) Can we beat the above? Try this new hybrid: T1 runs as BSP on 4 machines; T2 runs task-par. to T1 on the 5th machine. Then T3 runs as full BSP on all 5 machines. So, runtime now: $\text{Max}\{(2 + 12/3), 6\} + 2 + 30/4 = 6 + 2 + 7.5 = \mathbf{15.5!}$ This is the fastest hybrid. In general, the intuition is to ask if a task is “worth” paying the BSP overhead (for Mgr) vs should it be task-par. The longer-running a task, the more likely BSP is worth it for that.
- d. Task-par.: $\text{Max}\{12+30, 30+30\} = 60$. BSP will have only 1 worker; so it is not possible to beat 60.
- e. Task-par.: Still 60. Full BSP for all tasks has runtime: $2*3$ (Mgr) + $12/2 + 30/2 + 30/2 = \mathbf{42}$. Can the hybrid from question B be faster here? Its runtime will be: $\text{Max}\{12, 30\} + 2 + 30/2 = 47$. It is slower than full BSP now! The intuition for why this switch happened has to do with the length of T2 (x). By running T1 and T2 in task-par. manner when T2 is relatively so much more “long-running,” we are idling the worker running T1 too much. BSP ends up being better off.

12. Suppose you are given the following Gantt chart for a BSP data-parallel execution of two back-to-back processes on a 3-worker cluster (both have run to completion)



Suppose P1 & P2 take 15 & 30 units resp. when run on a single machine. What is the speedup seen by this workload?

Ans: $(15+30) / 30 = 1.5x$. This is because with one worker, no overhead of orchestration exists.

Tips on MR

General approach/advice on how to cast a data analytics computation onto the MapReduce API:

Step 1) Identify the exact data access pattern of the computation over the dataset. Draw it out to see it visually if you like.

Step 2) Identify how to decompose the bulk of the whole computation into a bunch of independent chunk computations on sub-elements (rows/ columns/tiles). Typically, scalability along rows is the most preferable because most modern large-scale datasets have large numbers of rows.

Step 3) Identify how to aggregate those decomposed parts to get the final result as if it was computed in a single-threaded in-RAM manner. This aggregation step may not always be needed though.

Step 4) Align the sharding with Step 2. Put the independent chunk computations in the Mapper. Identify what the Mapper's intermediate output (emit) data structure should be. Put the aggregation in Step 3 and any post processing in the Reducer.

13. Suppose you are given a large dataset with 50 numeric and 9 categorical features (domain size of 50 each). The HDFS file size is 3 TB.
 - a. Write pseudocode for (or just describe precisely in prose) MapReduce job(s) to compute this dataset's **correlation matrix**. Hint: It is OK to do it as 2 separate MapReduce jobs, although 1 suffices.

- b. What is the rough total disk I/O cost of the above in TB?
Include both reads and writes of intermediate data and output.

Ans: Problem solved in the document on Scaleable Data Access Problems.